



WAPI DLL

SOFTWARE GUIDE

WAPI DLL – WEIGHING APPLICATION PROGRAMMING INTERFACE

TENZOVÁHY, S.R.O.

DOCUMENT NUMBER: NAV19-005

VERSION: 2.5



TABLE OF CONTENTS

1	Document purpose and disclaimer.....	5
2	Technical specification	6
2.1	WAPI design rules.....	6
3	WAPI functions description.....	7
3.1	General and special functions.....	7
3.2	Connectivity functions	8
3.3	Functions for logging and managing user accounts	8
3.4	Weighing functions.....	9
3.5	Function for setting outputs of IOUNIT module	9
3.6	Functions for reading weighing records from DSD (Alibi) memory.....	9
4	WAPI application examples.....	10
4.1	Example 1 – default (static) weighing mode.....	10
4.1.1	Description of data message	10
4.2	Example 2 - Weighing of vehicles in motion	11
4.2.1	Connection to TCP server of WSERVER as TCP client	11
4.2.2	User Login.....	12
4.2.3	Start weighing-in-motion (WIM)	13
4.2.4	Obtaining data during weighing	13
4.2.5	Termination of weighing.....	14
4.2.6	Description of received message.....	15
4.3	Example 3 – Weighing of vehicles in static mode - axle by axle	16
4.3.1	Connection to TCP server of WSERVER as TCP client	16
4.3.2	User Login.....	17
4.3.3	Start weighing of vehicle in static mode - axle-by-axle	17
4.3.4	Obtaining data during weighing	18
4.3.5	Termination of weighing.....	18
4.3.6	DESCRIPTION OF RECEIVED MESSAGE.....	19
4.4	Example 3 - Reading weighing records from DSD memory	19
4.4.1	Reading records created in certain time interval	20
4.4.2	Reading records using indexes	21
4.5	Example 4 - Create new user	22
4.6	Example 5 – Delete existing user.....	23



4.7	Example 6 – Change password of existing user	24
4.8	Example 7 – list of existing users.....	24
5	Status and error codes	25
5.1	List of internal error codes.....	25
5.2	Weighing system status and error codes.....	26



1 DOCUMENT PURPOSE AND DISCLAIMER

This document describes WAPI (Weighing API) – unified application programming interface for communication with weighing systems produced by TENZOVÁHY, s.r.o.

Manufacturer TENZOVÁHY, s.r.o. is not responsible for any damages, losses, costs or expenses, direct, indirect or incidental, consequential or special, related to or resulting from the use of this product or other products listed in this document.



IMPORTANT NOTE

When developing software using the WAPI interface, it is recommended to respect the requirements for the operation of scales defined in these documents:

- *User Manual - available from the weighing system supplier or from the TENZOVÁHY, s.r.o.*
- *[International Recommendation OIML R 76](#) for weighing instruments with non-automatic operation (for static weighing)*
- *[International Recommendation OIML R 134](#) for weighing instruments with automatic operation (for weighing in motion)*
- *[WELMEC 7.2 Software guide](#)*



2 TECHNICAL SPECIFICATION

Weighing Application Program Interface - hereinafter referred to as “**WAPI**” is a uniform high-level interface allowing general and uniform connection of client applications to the unified weighing server WSERVER developed by company TENZOVÁHY, s.r.o.

WSERVER is a multiplatform server application developed in parallel for Microsoft Windows, Linux and Android operating systems. It is an essential part of legally relevant weighing software that provides evaluation of data from load cells and other weighing sensors, performs weight calculation and provides this result to connected client applications. WSERVER functions respect the international recommendations and standards for non-automatic weighing instruments OIML R 76, for automatic weighing instruments OIML R 134, the EU harmonized standard EN 45501 and the WELMEC Software Manual 7.2.

By communicating with WSERVER via the WAPI interface, the need to know the internal communication protocol between the weighing server and the clients and the need to react proactively to changes caused by further development of the communication protocol is eliminated.

In the development of WAPI, independence from the software platform was respected. For Windows operating systems, WAPI takes the form of a DLL, for Linux systems it takes the form of an SO (shared object). The library exports the set of functions described later in this document.

2.1 WAPI DESIGN RULES

For the clarity of code and to ensure maximum compatibility of the WAPI library with other development tools, the following rules are respected:

- a) All functions are designed in such a way that no run-out exception gets out of the calling routine (try-except treatment will be used). In case of an error, a non-zero numeric result will be returned (see next point)
- b) If this makes sense in the context, all functions return a number code (of the *Integer* type), with **zero** representing a **successful execution** and a **non-zero value** representing an **error code**. The text of the last error can be obtained by calling a separate function of type *getLastErrorMsg*.
- c) All exported functions are declared with directive “*stdcall*” (for windows), or “*cdecl*” (for linux).
- d) If it is necessary to pass (or return) a string argument to the exported function, the type *PAnsiChar* (or *PWideChar*) is always used. It will never use "open string" for arguments, i.e. a type from *string* / *ansistring* / *widestring*.
- e) All exported functions have a name beginning with a uniform *wapi* prefix, i.e. *wapi_get_last_error_msg*.



3 WAPI FUNCTIONS DESCRIPTION

All functions are synchronous. This means that if the function is implemented in the form of sending a packet with a request **#REQ** (which is the majority), the function waits for the response of the WSERVER weighing server (packet **#RES**) and only then returns with the appropriate error code, respectively with requested data.

With regard to the development of WAPI in the Delphi integrated development environment, the function declarations are given in the syntax of the Pascal language, while the *stdcall* directive, resp. *cdecl* directive is omitted. If the function returns an Integer value and is not specified otherwise, zero means success and anything else an error code.

3.1 GENERAL AND SPECIAL FUNCTIONS

function wapi_get_api_version_num: Integer;

Returns the version number of the interface for later compatibility. The default version returns **1**.

function wapi_get_last_error_msg: PAnsiChar;

Returns the text of the last error (exception). If none is available, it returns an empty string.

function wapi_get_last_error: Integer;

Returns the number of the last error encountered (exceptions). If none is available, it returns zero.

function wapi_register_message_callback(callback: Pointer): Integer;

Defines the procedure by which the weighing server will send messages to clients (**#MSG** packets). Unregistration can be done by entering the **nil** value.

WAPI calls a registered procedure whenever a client message arrives from the server. The procedure called will thus have the following declaration:

```
procedure (msginfo: PMSGINFO); stdcall;  
  
PMSGINFO = ^TMSGINFO;  
TMSGINFO = record  
    packet: PAnsiChar;  
end;
```



3.2 CONNECTIVITY FUNCTIONS

function wapi_connect(address: PAnsiChar; port: Integer): Integer;

Creates a connection to a weighing server WSERVER. The return value is the return value from the system socket.

function wapi_disconnect: Integer;

Closes the connection to the weighing server.

3.3 FUNCTIONS FOR LOGGING AND MANAGING USER ACCOUNTS

function wapi_login(username, password: PAnsiChar): Integer;

Logs the user on to the weighing server and sets the user level based on the user name and password entered.

function wapi_logout: Integer;

Logs out the current user and sets the connection with the lowest permission level (0 = boot).

function wapi_create_account(username, password, fullname: PAnsiChar; level: Integer): Integer;

Creates a user account on the server by sending a packet #REQ CMD: ACCOUNTCREATE.

function wapi_delete_account(username: PAnsiChar): Integer;

Deletes a user account from the server by sending a packet #REQ CMD:ACCOUNTDELETE.

function wapi_change_account(username, old-password, new-password: PAnsiChar): Integer;

Change the password for the user account by sending a packet #REQ CMD:ACCOUNTCHANGE.

function wapi_getaccounts(var list_user: PAnsiChar) : Integer;

Returns *list_user* variable containing a list of all existing users.



3.4 WEIGHING FUNCTIONS

function wapi_zero_weigh: Integer;

Sets the current load level as zero level by sending a packet #REQ CMD:ZERO.

function wapi_set_weigh_mode(mode,direction: Integer): Integer;

Switches the TENZOWIM 134 weighing instrument to the desired weighing mode.

mode:

- 0 Default static mode (sending packet #REQ CMD:SETMODE STAT); parameter *direction* is ignored
- 1 Vehicle weigh-in-motion mode (sending packet #REQ CMD:SETMODE DYNAM LEFT or RIGHT according to parameter *direction*)
- 2 Axle weigh-in-motion mode (sending packet #REQ CMD:SETMODE DYNAA LEFT or RIGHT according to parameter *direction*)
- 3 Static weighing of the vehicle sequentially by axles (the #REQ CMD:SETMODE STATW packet will be sent) the *direction* parameter is ignored

direction:

- 0 Weigh-in-motion weighing from left (LEFT)
 - 1 Weigh-in-motion weighing from right (RIGHT)
- 0 = Weigh-in-motion weighing from left (LEFT)
1 = Weigh-in-motion weighing from right (RIGHT)

3.5 FUNCTION FOR SETTING OUTPUTS OF IOUNIT MODULE

function wapi_setoutput_iounit(val : Integer): Integer;

Sets the binary outputs of the IOUNIT module. These are 6 binary values, where the output BOUT0 is set by (lowest) bit 0, BOUT1 by bit 1, etc.

3.6 FUNCTIONS FOR READING WEIGHING RECORDS FROM DSD (ALIBI) MEMORY

**function wapi_getrecords_date_time(date_from:PAnsiChar;
date_to:PAnsiChar): Integer;**

Creates a request to obtain existing weighing records that were created within the time interval specified by the *date_from* and *date_to* values. The records are then sent to the WSERVER client and can be selected using the *callback function* mentioned in Section 3.1.



```
function wapi_getrecords_idx(idx_from:Integer; idx_to:Integer) :  
Integer;
```

Creates a request to obtain existing weighing records that were created within the time interval specified by the *idx_from* and *idx_to* values. The records are then sent to the WSERVER client and can be selected using the *callback function* mentioned in Section 3.1.

4 WAPI APPLICATION EXAMPLES

The following examples illustrate the use of the WAPI library in practice, i.e. in a custom user application that uses the supplied DLL library.

4.1 EXAMPLE 1 – DEFAULT (STATIC) WEIGHING MODE

In a static mode, actual status of weighing system (and loading of weighbridge) is sent periodically (approx. in 0,1s interval) from WSERVER to the connected client.

This “default” weighing mode is enabled when the weighing system is switched on.

WSERVER send data to WAPI client if following steps a. and b. are performed, or if the system is switched to static mode by command c.:

- a. Connection with TCP server is established by *wapi_connect(IP_address, 65002)*,
- b. Callback function is registered *wapi_register_message_callback()*.
- c. Weighing mode is set to static by *wapi_set_weigh_mode(0,0)*.

Weighing system is automatically switched to static mode whenever weighing in motion is finished.

Client can retrieve data by *wapi_register_message_callback* function, see Section 3.1.

Data message example:

```
#MSG MODE:3;STATUS:0;BRIDGE1: STATE:6 WEIGHT:0;CHANNEL1: VALUE:401;CHANNEL2:  
VALUE:394;CHANNEL3: VALUE:401;CHANNEL4: VALUE: 396;CAL:0;TEMPDIG0:  
235;TEMPDIG1: 240;TEMP0: 23.5;TEMP1: 24.0;SD: 2035712;SDP: 0;WBC: 1;SUM: 0000;CRC:  
C8C16FE0;BININ: 0000;BINOUT: 0000;
```

4.1.1 DESCRIPTION OF DATA MESSAGE

STATUS	weighing result code. 0=OK, other result is the error code – see chapter 5.
MODE	weighing system status (0=out of service, 1 = initializing, 2 = sampling stopped, 3 = overload, 4 = unstable load, 5 = stable load, 6 = zero)
STATE	weighbridge status (0=out of service, 1=error, 2=autocalibration, 3=overload, 4=unstable load, 5=stable load, 6=zero)



WEIGH	weight in (kg) rounded to scale interval.
CHANNEL1-4: VALUE:XXX	output of load cell connected to channel (1-4) in [digits]
CAL	Indication of calibration mode (0/1).
TEMPDIG0,1	output of external thermo sensors in [dig]
TEMP0, 1	output of external thermo sensors in [°C].
SD	storage media occupancy by ALIBI.DAT file in [bytes]
SDP	storage media occupancy by ALIBI.DAT file [%]
WBC	number of records in ALIBI.DAT.
SUM	CRC security error code of WSERVER LRS and ALIBI.DAT LRD. bit0 ...0/1 is not / is a CRC error of WSERVER legally relevant software bit1 ...0/1 is not / is a CRC error of ALIBI.DAT legally relevant data
CRC	cyclic checksum to ensure the integrity of WSERVER and LRD data
BININ	status of binary inputs of IOUNIT module (hexadecimal, 2 bytes)
BINOUT	status of binary inputs of IOUNIT module (hexadecimal, 2 bytes)

4.2 EXAMPLE 2 - WEIGHING OF VEHICLES IN MOTION

The following example illustrates the use of the WAPI library for weighing vehicles in motion.

4.2.1 CONNECTION TO TCP SERVER OF WSERVER AS TCP CLIENT

wapi_connect(IP_address, 65002),

where **IP_address** is address of Indication unit with WSERVER running

Example in Delphi language:

```
...  
function Connect(address: PAnsiChar; port: Integer): Integer; stdcall;  
    external 'Wapi.dll' name 'wapi_connect';  
...  
var R : Integer;  
    Pch : PAnsiChar;  
    s : String;
```



```
s2 : String;  
begin  
s:=elp.Text;  
SetLength(s2, Length(s)+1);  
Pch := @s2[1];  
StrPCopy(Pch, AnsiString(s));  
R:=Connect(Pch, StrToInt(ePort.Text));  
if R=0 then  
  ShowMessage('Connection Error')  
else  
  ShowMessage('Connection OK');  
end;
```

4.2.2 USER LOGIN

wapi_login(Username, Password),

where **Username** is existing user of on WSERVER and **Password** is his valid password.

Example in Delphi language:

```
var s : String;  
    PName : PAnsiChar;  
    PPsw : PAnsiChar;  
    i : Integer;  
begin  
s:='robert';  
PName:=@s[1];  
StrPCopy(PName, AnsiString(s));  
s:='prag';  
PPsw:=@s[1];  
StrPCopy(PPsw,AnsiString(s));  
i:=wapi_login(PName, PPsw);  
showmessage(IntToStr(i));  
end;
```

After calling the **wapi_login** function, the DLL library sends a message to WSERVER:

#REQ CMD:LOGIN UID:robert pwd:prag;

Reply from WSERVER:



#RES RESULT:OK;ACCOUNT UID:robert LEVEL:4 NAME:"fullname";#ENDRES,

where on position *fullname* is name and surname of the logged user.

4.2.3 START WEIGHING-IN-MOTION (WIM)

wapi_set_weigh_mode(1, direction),

where *direction* is set according to driving direction as described above

Example in Delphi language:

```
var i : Integer;  
dir : integer  
begin  
  dir:=1;  
  i:=wapi_set_weigh_mode(1, dir);  
  ShowMessage(IntToStr(i));  
end;
```

After calling the **wapi_set_weigh_mode** function, the DLL library sends a message to WSERVER:

#REQ CMD:SETMODE DYNAV RIGHT;

Reply from WSERVER:

#RES RESULT:OK;#ENDRES

4.2.4 OBTAINING DATA DURING WEIGHING

The pre-registered function *wapi_register_message_callback* returns data. It waits from the beginning of the vehicle crossing over the weighing platform (weighbridge) for the timeout that is visible in the currently received *callback* function:

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:14;

The TIMEOUT value decreases after one second and the first axle must reach the weighbridge before it counts down to zero. Weighing is terminated either by this adjustable TIMEOUT or by switching to static mode. When the first axle is detected, the following message is sent from WSERVER (example):

**#MSG MODE:5;DISPLAY: DIR:1 AXLE:1 LOAD:0 OVERLOAD:0 LASTWEIGHT:5777 TOTAL:5777
TIMEOUT:14;**



From this moment the TIMEOUT value decreases every second and next axles are expected to pass before the TIMEOUT value drops to zero. Detection of the next axle creates a similar message:

#MSG MODE:5;DISPLAY: DIR:1 AXLE:2 LOAD:0 OVERLOAD:0 LASTWEIGHT:10361 TOTAL:16138 TIMEOUT:14;

NOTE: Messages are generated periodically 10 times per second. Data stream sent in one second looks as in the following example:

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:14;

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:14;

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:14;

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:14;

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:14;

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:14;

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:14;

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:14;

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:14;

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:14;

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:13;

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:13;

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:13;

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:13;

#MSG MODE:5;DISPLAY: DIR:1 AXLE:0 LOAD:0 OVERLOAD:0 LASTWEIGHT:0 TOTAL:0 TIMEOUT:13;

4.2.5 TERMINATION OF WEIGHING

Weighing of the vehicle in motion is terminated either by expiry of the above **TIMEOUT** or by sending a message to WSERVER using the **wapi_set_weigh_mode** function - set to static mode.

Example in Delphi language:

```
var i : integer;  
begin  
    i:=wapi_set_weigh_mode(0,0);
```



```
ShowMessage(IntToStr(i));  
end;
```

The first parameter of the function determines the *static weighing mode*, the second parameter is the *drive direction* of the vehicle, whose value is not significant in this case.

After calling **wapi_set_weigh_mode** function the command is sent to WSERVER:

```
#REQ CMD:SETMODE STAT;
```

Reply from WSERVER:

```
#RES RESULT:OK;#ENDRES
```

The pre-registered function **wapi_register_message_callback** returns the weighing status data after the weighing is finished.

Weighing is finished either by an adjustable timeout or by switching to Static mode. Using *callback* function, we can receive, for example, the following message:

```
#MSG OBJECT:VEHICLE RES:0 RECID:7 TIME:"2020-01-15 11:21:55" DIR:0  
WEIGHT:4140 SPEED:4.99 INFOTEXT:"" WS:"0" AXLECOUNT:2 AXLENO:1  
WEIGHT:2760 GWEIGHT:2755 WHEEL1:1378 WHEEL2:1378 BASE:1.84 SPEED:4.82  
AXLENO:2 WEIGHT:1380 GWEIGHT:1377 WHEEL1:688 WHEEL2:688 BASE:0.00  
SPEED:5.17;
```

4.2.6 DESCRIPTION OF RECEIVED MESSAGE

RES	introduces responses sent by the WSERVER.
RECID	unique number, under which the data record is stored by WSERVER in DSD (Alibi) memory of the Indication unit.
TIME:	Date and time when weighing record was created.
DIR	vehicle driving direction (0=from left, 1=from right).
WEIGHT	total vehicle weight (GVW) in [kg]; value rounded to weighing increment „d“.
SPEED	vehicle speed in [km/h].
INFOTEXT	Optional text that can be set when weighing is started
WS	weighing system identifier number



AXLECOUNT	number of vehicle axles (or vehicle combination axles).
AXLENO	number (order) of the axle, to which the following information refers:
WEIGHT	axle weight (load) in [kg]; rounded to weighing increment „d“.
GWEIGHT	unrounded axle weight in [kg].
WHEEL1 (2)	wheel weight on certain axle in [kg].
BASE	wheelbase in [m], it is distance to following axle, so it is always 0.00 for the last axle
SPEED	axle speed in [km/h].

4.3 EXAMPLE 3 – WEIGHING OF VEHICLES IN STATIC MODE - AXLE BY AXLE

The example illustrates the use of the WAPI library for weighing a vehicle, which is weighed by stopping its individual axles on the weighbridge one by one.

4.3.1 CONNECTION TO TCP SERVER OF WSERVER AS TCP CLIENT

wapi_connect(IP_adress, port),

kde *IP_adresa* je adresa indikační jednotky, na níž běží WSERVER a *port* je číslo portu (obvykle 65002)

Exmple in Delphi language:

```
...
functionConnect(address: PAnsiChar; port: Integer): Integer; stdcall;
external 'Wapi.dll' name 'wapi_connect';
...
var R :Integer;
Pch : PAnsiChar;
s :String;
s2 : String;
begin
s:=elp.Text;
SetLength(s2, Length(s)+1);
Pch := @s2[1];
StrPCopy(Pch, AnsiString(s));
R:=Connect(Pch, StrToInt(ePort.Text));
if R=0 then
ShowMessage('ConnectionError')
```




```
else  
  ShowMessage('Connection OK');  
end;
```

4.3.2 USER LOGIN

wapi_login(Username, Password),

where **Username** is existing user of on WSERVER and **Password** is his valid password.

Example in Delphi language:

```
var s:String;  
PName:PAnsiChar;  
PPsw:PAnsiChar;  
i:Integer;  
begin  
  s:='robert';  
PName:=@s[1];  
StrPCopy(PName, AnsiString(s));  
  s:='prag';  
PPsw:=@s[1];  
StrPCopy(PPsw, AnsiString(s));  
  i:=wapi_login(PName, PPsw);  
  showmessage(IntToStr(i));  
end;
```

After calling the **wapi_login** function, the DLL library sends a message to WSERVER:

#REQ CMD:LOGIN UID:robert pwd:prag;

Reply from WSERVER:

#RES RESULT:OK;ACCOUNT UID:robert LEVEL:4 NAME:"fullname";#ENDRES,

where on position *fullname* is name and surname of the logged user.

4.3.3 START WEIGHING OF VEHICLE IN STATIC MODE - AXLE-BY-AXLE

To start the weighing, the logged-in user must have permissions with the user level, i.e. with a value of at least 1

wapi_set_weigh_mode(3, 0),



where the second parameter (value 0) has no meaning in this mode.

Example in Delphi language:

```
var i :Integer;
dir : integer
begin
dir:=1;
i:=wapi_set_weigh_mode(3, 0);
ShowMessage(IntToStr(i));
end;
```

After calling the **wapi_set_weigh_mode** function, the DLL library sends a message to WSERVER:

#REQ CMD:SETMODE STATW;

Reply from WSERVER:

#RES RESULT:OK;#ENDRES

4.3.4 OBTAINING DATA DURING WEIGHING

The pre-registered **wapi_register_message_callback** function returns data. It waits until a message starting with "**#MSG OBJECT:**" is received.

The callback function can be used to receive data:

#MSG MODE:8;STATUS:0;BRIDGE1: STATE:4 WEIGHT:340;CHANNEL1:.....

#MSG MODE:8;STATUS:0;BRIDGE1: STATE:4 WEIGHT:640;CHANNEL1:

4.3.5 TERMINATION OF WEIGHING

Pre-registered function **wapi_register_message_callback** returns data after finishing of weighing, what is done when the scale is loaded and stable. For example, we will receive a message like this:

**#MSG OBJECT:STATIC RES:0 RECID:25 TIME:"2025-06-24 11:16:14" WEIGHT:3060 WS:"0"
PLATFORMCOUNT:2 PLATFORMNO:1 WEIGHT:2050; PLATFORMNO:2 WEIGHT:1010;**

Weighing can be interrupted at any time by sending a message switching to the default static mode.



Example in Delphi language:

```
var i :integer;  
begin  
  i:=wapi_set_weigh_mode(0,0);  
  ShowMessage(IntToStr(i));  
end;
```

The first function parameter determines the static mode, the second is the direction of travel of the vehicle, the value of which is not significant in this case.

When the **wapi_set_weigh_mode** function is called, the DLL sends a message to WSERVER:

#REQ CMD:SETMODE STAT;

WSERVER reply:

#RES RESULT:OK;#ENDRES

4.3.6 DESCRIPTION OF RECEIVED MESSAGE

RES	introduces responses sent by the WSERVER.
RECID	unique number, under which the data record is stored by WSERVER in DSD (Alibi) memory of the Indication unit.
TIME	Date and time when weighing record was created.
WEIGHT	total vehicle weight (GVW) in [kg]; value rounded to weighing increment „d“.
WS	weighing system identifier number
PLATFORMCOUNT	number of attached weighing platforms
PLATFORMMNO	platform number, followed by the load on each platform (i. e.: PLATFORMMNO:1 WEIGHT:2050; PLATFORMMNO:2 WEIGHT:1010;)

4.4 EXAMPLE 3 - READING WEIGHING RECORDS FROM DSD MEMORY

Client must be connected (see section 4.2.1) and logged (see section 4.2.2) with level “user” (1) or higher to be authorized for reading records from internal DSD (Alibi) memory.



4.4.1 READING RECORDS CREATED IN CERTAIN TIME INTERVAL

Function **wapi_getrecords_date_time** with *parameters of time interval* is used.

Example in Delphi language:

```
var p_str_od : PAnsiChar;
    p_str_do : PAnsiChar;
    s : String;
    i: Integer;
begin
    s:=eFrom.Text;
    p_str_od:=@s[1];
    StrPCopy(p_str_od,AnsiString(s));
    s:=eTo.Text;
    p_str_do:=@s[1];
    StrPCopy(p_str_do,AnsiString(s));
    i:= wapi_getrecords_date_time(p_str_od, p_str_do);
    ShowMessage(IntToStr(i));
end;
```

Where:

Time format is:	YYYY-MM-DD HH:MM:SS
<i>eFrom.Text</i> is for example	2018-01-01 00:00:00
<i>eTo.Text</i> is for example	2020-12-31 00:00:00

When the **wapi_getrecords_date_time** function is called, the DLL sends a message to WSERVER:

#REQ CMD:GETRECORDS DATEFROM: "2018-01-01 00:00:00 " DATETO: " 2020-12-31 00:00:00 ";

WSERVER reply with data to the client that has to be captured by above mentioned and registered *callback* function - example:

#RES RESULT:OK;RECORD RECID:1 TYPE:0 RES:0 TIME:"2020-01-15 11:07:47" DIR:1 WEIGHT:5510 SPEED:4.11 INFOTEXT:"" WS:"0" AXLECOUNT:2 AXLENO:1 WEIGHT:2755 GWEIGHT:2755 WHEEL1:1378 WHEEL2:1378 BASE:1.95; AXLENO:2 WEIGHT:2755 GWEIGHT:2755 WHEEL1:1378 WHEEL2:1378 BASE:0.00;

RECORD RECID:2 TYPE:0 RES:0 TIME:"2020-01-15 11:08:19" DIR:1 WEIGHT:11018 SPEED:3.76 INFOTEXT:"" WS:"0" AXLECOUNT:5 AXLENO:1 WEIGHT:2755 GWEIGHT:2755 WHEEL1:1378 WHEEL2:1378 BASE:1.55; AXLENO:2 WEIGHT:2755 GWEIGHT:2755 WHEEL1:1378 WHEEL2:1378 BASE:1.74; AXLENO:3 WEIGHT:1377 GWEIGHT:1377 WHEEL1:688 WHEEL2:688 BASE:1.99; AXLENO:4 WEIGHT:2754 GWEIGHT:2754 WHEEL1:1377 WHEEL2:1377 BASE:1.70; AXLENO:5 WEIGHT:1377 GWEIGHT:1377 WHEEL1:688 WHEEL2:688 BASE:0.00;



RECORD RECID:3 TYPE:0 RES:0 TIME:"2020-01-15 11:08:52" DIR:1 WEIGHT:6885 SPEED:3.91
INFOTEXT:"" WS:"0" AXLECOUNT:3 AXLENO:1 WEIGHT:2754 GWEIGHT:2754 WHEEL1:1377
WHEEL2:1377 BASE:1.60; AXLENO:2 WEIGHT:1377 GWEIGHT:1377 WHEEL1:688 WHEEL2:688
BASE:1.67; AXLENO:3 WEIGHT:2754 GWEIGHT:2754 WHEEL1:1377 WHEEL2:1377 BASE:0.00;

RECORD RECID:4 TYPE:0 RES:0 TIME:"2020-01-15 11:09:20" DIR:1 WEIGHT:13775 SPEED:3.77
INFOTEXT:"" WS:"0" AXLECOUNT:5 AXLENO:1 WEIGHT:4132 GWEIGHT:4132 WHEEL1:2066
WHEEL2:2066 BASE:1.45; AXLENO:2 WEIGHT:2755 GWEIGHT:2755 WHEEL1:1378 WHEEL2:1378
BASE:1.77; AXLENO:3 WEIGHT:2755 GWEIGHT:2755 WHEEL1:1378 WHEEL2:1378 BASE:1.41;
AXLENO:4 WEIGHT:1378 GWEIGHT:1378 WHEEL1:689 WHEEL2:689 BASE:1.68; AXLENO:5
WEIGHT:2755 GWEIGHT:2755 WHEEL1:1378 WHEEL2:1378 BASE:0.00;

#ENDRES

The example above show 4 weighing records created in desired time range.

Explanation of tags is in the section 4.2.6.

Note: finalized weighing records stored in DSD (Alibi) memory does not contain individual axle speeds, only the speed of vehicle.

4.4.2 READING RECORDS USING INDEXES

Function **wapi_getrecords_idx** with parameters of *start* and *finish index* is used for data reading.

Example in Delphi language:

```
var i : Integer;  
    IdxFrom : Integer;  
    IdxTo : Integer;  
begin  
    IdxFrom:=1;  
    IdxTo:=3;  
    i:=wapi_getrecords_idx(IdxFrm, IdxTo);  
    ShowMessage(IntToStr(i));  
end;
```

When the **wapi_getrecords_idx** function is called, the DLL sends a message to WSERVER:

#REQ CMD:GETRECORDS IDFROM:2 IDTO:3;

WSERVER reply with data to the client that has to be captured by above mentioned and registered *callback* function - example:

RECORD RECID:2 TYPE:0 RES:0 TIME:"2020-01-15 11:08:19" DIR:1 WEIGHT:11018 SPEED:3.76
INFOTEXT:"" WS:"0" AXLECOUNT:5 AXLENO:1 WEIGHT:2755 GWEIGHT:2755 WHEEL1:1378
WHEEL2:1378 BASE:1.55; AXLENO:2 WEIGHT:2755 GWEIGHT:2755 WHEEL1:1378 WHEEL2:1378



BASE:1.74; AXLENO:3 WEIGHT:1377 GWEIGHT:1377 WHEEL1:688 WHEEL2:688 BASE:1.99; AXLENO:4
WEIGHT:2754 GWEIGHT:2754 WHEEL1:1377 WHEEL2:1377 BASE:1.70; AXLENO:5 WEIGHT:1377
GWEIGHT:1377 WHEEL1:688 WHEEL2:688 BASE:0.00;

RECORD RECID:3 TYPE:0 RES:0 TIME:"2020-01-15 11:08:52" DIR:1 WEIGHT:6885 SPEED:3.91
INFOTEXT:"" WS:"0" AXLECOUNT:3 AXLENO:1 WEIGHT:2754 GWEIGHT:2754 WHEEL1:1377
WHEEL2:1377 BASE:1.60; AXLENO:2 WEIGHT:1377 GWEIGHT:1377 WHEEL1:688 WHEEL2:688
BASE:1.67; AXLENO:3 WEIGHT:2754 GWEIGHT:2754 WHEEL1:1377 WHEEL2:1377 BASE:0.00;

#ENDRES

The example above show 2 weighing records created in desired index range.

Explanation of tags is in the section 4.2.6.

Note: finalized weighing records stored in DSD (Alibi) memory does not contain individual axle speeds, only the speed of vehicle.

4.5 EXAMPLE 4 - CREATE NEW USER

To create a new user, we use the **wapi_create_account** function whose parameters are: *username*, *password*, *full name*, and *permission level*.

Example in Delphi language:

```
var
  i      : Integer;
  PName  : PAnsiChar;
  PPsw   : PAnsiChar;
  PFulName : PAnsiChar;
  sName  : String;
  sPsw   : String;
  sFulName : String;
begin

  sName:='robert';
  PName:=@sName[1];
  StrPCopy(PName, AnsiString(sName));
  sPsw:='prag';
  PPSW:=@sPsw[1];
  StrPCopy(PPsw, AnsiString(sPsw));
  sFulName:='robert cvek';
  PFulName:=@sFulName[1];
  StrPCopy(PFulName, AnsiString(sFulName));
```



```
Level:=1;  
i:= wapi_create_account(PName, PPSw, PFulName, Level);  
ShowMessage(IntToStr(i));  
except  
ShowMessage('Error filling in data');  
end;  
end;
```

In the example above is created user with permission level 1 - „user“

When the **wapi_create_account** function is called, the DLL sends a message to WSERVER:

#REQ CMD:ACCOUNTCREATE UID:robert PWD:“prag“ LEVEL:1 NAME:robert cvek;

WSERVER reply with following data:

#RES RESULT:OK;ACCOUNT UID:robert LEVEL:1 NAME:"robert cvek";#ENDRES

4.6 EXAMPLE 5 – DELETE EXISTING USER

To delete an existing user we use the **wapi_delete_account** function whose parameter is the *name* of the existing user (not the fullname).

Example in Delphi language:

```
var  
  i      : Integer;  
  Pname  : PAnsiChar;  
  s      : String;  
begin  
  s:='robert';  
  Pname:=@s[1];  
  StrPCopy(Pname, AnsiString(s));  
  i:=wapi_delete_account(PJmeno);  
  ShowMessage(IntToStr(i));  
end;
```

When the **wapi_delete_account** function is called, the DLL sends a message to WSERVER:

#REQ CMD:ACCOUNTDELETE UID:robert;

WSERVER reply with following data:

#RES RESULT:OK;#ENDRES



4.7 EXAMPLE 6 – CHANGE PASSWORD OF EXISTING USER

To change the password of an existing user, we use the **wapi_change_account** function whose parameters are: *user name*, *current password*, and *new password*.

Example in Delphi language:

```
var i      : Integer;
    PName  : PAnsiChar;
    PPsw   : PAnsiChar;
    s_name : String;
    s_new_psw : String;
    s_psw  : String;
    PNewPsw : PAnsiChar;
begin
    s_name:='robert';
    PName:=@s_name[1];
    StrPCopy(PName, AnsiString(s_name));
    s_psw:='prag';
    PPsw:=@s_psw[1];
    StrPCopy(PPsw, AnsiString(s_psw));
    s_new_psw:='pragnew';
    PNewPsw:=@s_new_psw[1];
    StrPCopy(PNewPsw, AnsiString(s_new_psw));
    i:= wapi_change_account(PName, PPsw, PNewPsw);
    ShowMessage(IntToStr(i));
end;
```

When the **wapi_change_account** function is called, the DLL sends a message to WSERVER:

#REQ CMD:ACCOUNTCHANGE UID:robert OLDPWD:prag NEWPWD:pragnew;

WSERVER reply with following data:

#RES RESULT:OK;#ENDRES

4.8 EXAMPLE 7 – LIST OF EXISTING USERS

To get a list of users, use the **wapi_getaccounts** function whose parameter is a variable with user names separated by a space.

Example in Delphi language:

```
var i : Integer;
```




```
users : PAnsiChar;  
str_users : String;  
begin  
  i:=wapi_getaccounts(users);  
  if i=0 then  
    begin  
      str_users:=String(StrPas(users));  
      ShowMessage(str_users);  
    end  
  else  
    ShowMessage(IntToStr(i));  
end;
```

When the **wapi_getaccounts** function is called, the DLL sends a message to WSERVER:

#REQ CMD:GETACCOUNTS;

WSERVER reply with following data:

#RES RESULT:OK;ACCOUNTS:a,b,power,robert,u;#ENDRES

Variable *users* include data:

a b power robert u

5 STATUS AND ERROR CODES

The **wapi_get_last_error_msg** and **wapi_get_last_error** functions return values corresponding to the error lists from the WSERVER weighing server, internal errors of the DLL itself, and also from synapses.

5.1 LIST OF INTERNAL ERROR CODES

```
30001 : 'Error convert IP address';  
30002 : 'Error TCP Connect';  
30003 : 'Error TCP Disconnect';  
30004 : 'Error callback registr';  
30005 : 'Error login timeout';  
30006 : 'General communication error';  
30007 : 'Error logout timeout';  
30008 : 'Error create account timeout';
```



30009 : 'Error delete account timeout';
30010 : 'Error change account timeout';
30011 : 'Error zero timeout';
30012 : 'Weigh mode error';
30013 : 'Weigh mode timeout';

5.2 WEIGHING SYSTEM STATUS AND ERROR CODES

The table below gives an overview of the possible weighing system states sent by the indicating unit in these MSG data messages:

- During static weighing under the STATUS tag
- In the weighing result under the RES tag

The weighing system status is also stored in the ALIBI.DAT data file under the ER header.

Error codes are divided into three groups as in the table below:

Code	Status code description	Hardware Error	Data Protection Error	Operating Error
0	OK	-	-	-
1	Impossible to start WIM weighing	X		
2	Vehicle with too many axles			X
3	Weighing terminated manually			X
4	Vehicle did not come in time limit			X
5	Vehicle with only one axle			X
6	Underspeeding vehicle (buffer overflow)			X
7	Vehicle stopped on weighbridge (no return to zero)			X
8	Overspeeding vehicle			X
9	Weighing out of tolerance			X
10	Load over weighing range			X
11	Load under weighing range			X
12	Weighbridge 1 is not empty			X
13	Weighbridge 2 is not empty			X
14	Weighbridge is not empty			X
32	Low zero level	X		
33	High zero level	X		
34	Communication error (no samples received)	X		



38	Weighing system in wrong mode			X
39	Buffer overflow	X		
40	Buffer underflow	X		
41	High side unevenness of load (big difference between left and right side load)			X
42	Temperature out of range	X		
43	Insufficient permission level for the operation			X
44	CRC error	X	X	
45	Packet timestamp mismatch error		X	
46	Packet order mismatch error		X	
47	Time for command execution elapsed			X
48	No control data available	X		
49	Calibration switch is not active			X
99	Other unspecified error	X		